

Lumenwild: The Five Frontiers

Project Summary and Technical Report

Executive Summary

Lumenwild is an offline, asset-free 2D side-scrolling platformer prototype built in Python with Pygame. The player controls a knight through five themed worlds, collects items, defeats enemies, reaches stage goals, and eventually faces a final boss. The game emphasizes deterministic procedural content, responsive platforming, simple combat, progression, and a renderer made entirely from Pygame primitives.

Current verified status: the game starts successfully with pygame-ce 2.5.7 on Python 3.14.2, and the automated test suite passes all four tests.

Project Goals

- Create a playable platformer without external art, sound files, network access, or asset downloads.
- Provide a compact campaign structure with five worlds and repeatable stages.
- Combine movement precision, combat, collectibles, hazards, checkpoints, and escalating difficulty.
- Persist completion and coin totals in a local JSON save file.
- Keep audio optional so gameplay remains usable when a mixer or audio device is unavailable.

Gameplay and Player Experience

- Title screen leads to a world and level selection map.
- Movement supports A/D or arrow keys; jumping supports Space, W, or Up.
- Z/X attacks with a short active hitbox; projectile powerups enable ranged attacks.
- Dash becomes available in later worlds; speed and double-jump powerups change movement behavior.
- The player has five health points, invincibility frames after damage, shield protection, coyote time, and jump buffering.
- Coins, gems, relics, checkpoints, powerups, spikes, falling platforms, disappearing platforms, enemy projectiles, and a goal structure each stage.
- Pause, restart, map access, keyboard input, joystick input, and basic touch zones are supported.

Worlds and Content

World	Theme	Visual identity
1	Emerald Canopy	Forest / meadow greens
2	Sunken Sunspire	Desert oranges and teal accents
3	Frostline Peaks	Ice blues and snow tones
4	Hollowstone Depths	Cave purples and dark stone
5	Cinder Crown	Volcanic reds and golds

The active campaign generator creates 15 intended stages: three stages across each of the five worlds. Stage width grows with stage and world number, platforms are placed from deterministic coordinate rules, and enemy counts and difficulty scale with progression.

Technical Architecture

File	Responsibility
main.py	Application entry point, Pygame loop, menus, input, camera, collision interactions, scoring, HUD, stage transitions, and
player.py	Player rectangle, physics, movement, jump buffering, coyote time, dash, attacks, projectiles, damage, powerups, and
enemy.py	Enemy state and movement behaviors for walker, flyer, charger, thrower, ambusher, armored, and phantom variants.
level.py	Tile grid loading/generation, colliders, hazards, dynamic tiles, stage entities, and tile rendering.
constants.py	Screen, physics, player, timing, color, control, and world-theme constants.
progression.py	JSON-backed completion and coin persistence.
audio.py	Procedural one-shot sound generation with graceful mixer failure handling.
data/	Authored JSON level example with layout and entity placement.
tests/	Unit tests for enemy movement and player jump behavior.

Core Systems

Physics and collision: The player is a 32 by 48 Pygame rectangle. Gravity is capped by terminal velocity, horizontal motion uses acceleration-like direct speed and friction, and movement is resolved separately on each axis against tile rectangles.

Combat: Attacks create a short-lived hitbox in front of the player. Ranged projectiles are available through a powerup. Enemies can be defeated with attacks, projectiles, or downward stomps. Damage applies temporary invincibility and may respawn the player at the last checkpoint.

Enemy behavior: Enemies patrol, chase nearby players, fly vertically, charge, throw telegraphed projectiles, ambush vertically, or teleport as phantoms. Health and speed scale with world and stage difficulty. The final armored boss has increased health and progressively more projectiles.

Level system: Levels use a two-dimensional integer grid: empty, floor, spike, falling, and disappearing tiles. JSON levels can be loaded, while campaign stages are generated from world and stage indexes. Dynamic tiles deactivate after the player stands on them for a configured duration.

Rendering: Sprites, backgrounds, clouds, trees, terrain, collectibles, UI, and effects are drawn procedurally with Pygame primitives. World 1 has a layered parallax forest background; later worlds use theme-specific sky and ground colors.

Persistence: Progress stores completed world-stage pairs and total coins in `lumenwild_save.json`. The current save contains two completed stages and 75 coins. Save failures are intentionally ignored so the game can still run.

Controls

Action	Keyboard / other input
Move	A/D or Left/Right arrows; joystick axis
Jump	Space, W, Up, joystick button, or touch zone
Attack	Z or X; joystick button
Dash	Left Shift after unlocking dash
Pause / resume	Escape
Restart	R
Map	M during play
Background style	1, 2, or 3

Testing and Verification

The command `python -m unittest discover -s tests -v` was run with the configured Python 3.14.2 interpreter. All four tests passed: two enemy movement tests and two player jump tests. The game was also launched successfully after installing `pygame-ce 2.5.7`, which provides the `pygame` module for this Python version.

Verified test cases:

- Enemy patrol stays within its movement bounds and reverses direction.
- Enemy moves toward a nearby player.
- Jump only starts from ground contact and uses the configured jump velocity.
- Jump input is buffered when pressed before landing.

Current Limitations and Maintenance Notes

- The repository contains a small older Player demo in game.py and a legacy demo block retained in main.py; the active executable path is main.py importing player.py.
- The map renders four level buttons, but the main campaign completion logic advances after three stages per world. This is a UI/content consistency issue to resolve if the fourth button is not intentional.
- The automated tests focus on movement and jumping; collision resolution, combat, boss phases, persistence, rendering, and menu transitions do not yet have automated coverage.
- pygame-ce is currently the practical dependency for Python 3.14. The original pygame package attempted a source build and failed because its build process expects removed distutils components.
- The project has no dependency lockfile or requirements file. Adding one would make setup more reproducible.

Run Instructions

From the project folder:

```
python main.py
```

Recommended dependency for the verified environment: pygame-ce. Close the game window to stop the process. Progress is written beside the game when the operating system permits file writes.

Overall Assessment

Lumenwild is a substantial playable prototype with a coherent game loop and a strong amount of systems work for an asset-free project. Its strongest qualities are the procedural presentation, broad gameplay feature set, deterministic stage generation, graceful audio fallback, and simple persistence model. The next practical development step is to consolidate the legacy code, align the level-selection UI with the intended three-stage campaign, and expand automated tests around the highest-risk gameplay interactions.